

String

- Object class `String` represents a string of characters.
- Belongs to the `java.lang` package
- String has constructors and methods
- String has two operators, `+` and `+=` for concatenation

Examples

```
//assign a literal to a String variable  
String name = "Robert";
```

```
//calling a method on a literal String  
char firstInitial = "Robert".charAt(0);
```

```
//calling a method on a String variable  
char firstInitial = name.charAt(0);
```

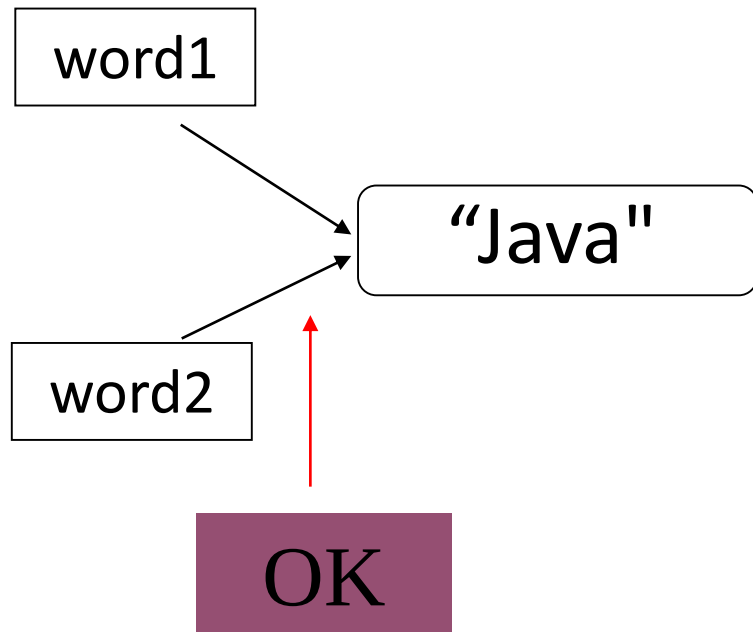
Immutability

- Once created, a string cannot be changed
- Such objects are called **immutable**

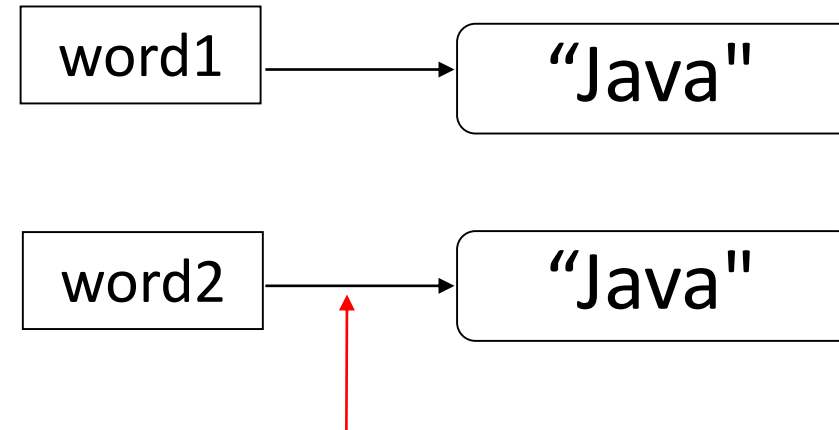
Advantages of Immutability

- Uses less memory.

```
String word1 = "Java";  
String word2 = word1;
```



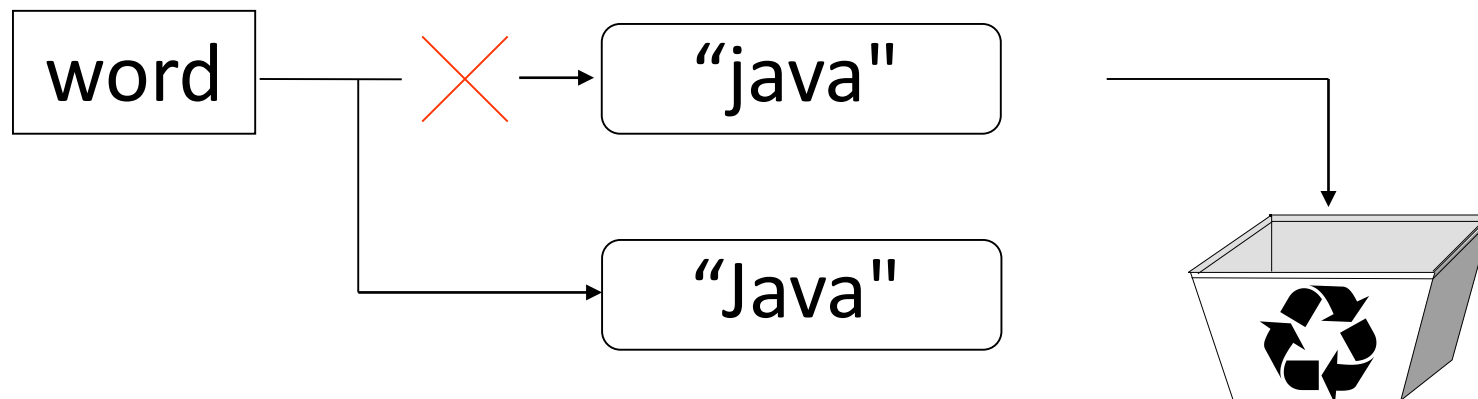
```
String word1 = "Java";  
String word2 = new String(word1);
```



Disadvantages of Immutability

- Less efficient — you need to create a new string and throw away the old one even for small changes.

```
String word = "Java";  
char ch = Character.toUpperCase(word.charAt (0));  
word = ch + word.substring (1);
```

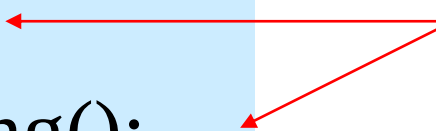


Empty Strings

- An empty String has no characters
- It's length is 0

```
String word1 = "";  
String word2 = new String();
```

Empty strings



- Not the same as an uninitialized String

```
private String errorMsg;
```

**errorMsg
is null**



Methods — length, charAt

`int length();`

- Returns the number of characters in the string

`char charAt(i);`

- Returns the char at position i.



Character positions in strings are numbered starting from 0 – just like arrays.

`"Problem".length();`

`"Window".charAt (2);`

Returns:

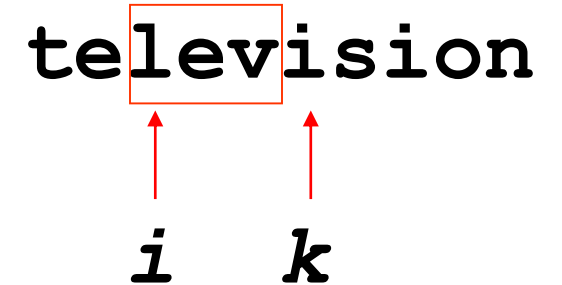
7

'n'

Methods — substring

Returns a new String by copying characters from an existing String.

String subs = word.**substring** (i, k);
returns the substring of chars in
positions from **i** to **k-1**

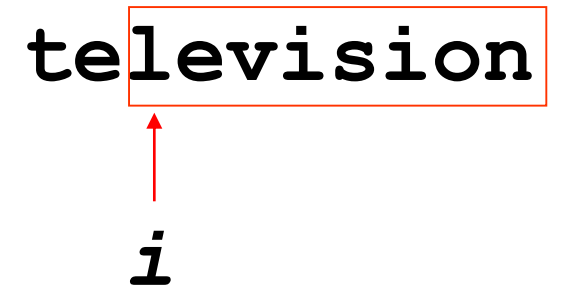


television

i *k*

The diagram shows the word "television" with a red box highlighting the characters "lev". Two red arrows point upwards from the labels *i* and *k* to the start and end of the boxed substring, respectively.

String subs = word.**substring** (i);
returns the substring from the **i**-th char
to the end



television

i

The diagram shows the word "television" with a red box highlighting the entire word. A red arrow points upwards from the label *i* to the start of the boxed substring.

```
"television".substring (2,5);  
"immutable".substring (2);  
"bob".substring (9);
```

→ "lev"
→ "mutable"
→ "" (empty string)

Methods — Concatenation

```
String word1 = “re”, word2 = “think”; word3 = “ing”;
```

```
int num = 2;
```

```
String result = word1 + word2;
```

```
String result = word1.concat (word2);
```

```
result += word3;
```

```
result += num;
```

Methods — indexOf(character)

0 2 6 10 15

String date = "President George Washington";

date.indexOf ('P'); 0

date.indexOf ('e'); 2

date.indexOf ("George"); 10

date.indexOf ('e', 3); 6

date.indexOf ("Bob"); -1

date.lastIndexOf ('e'); 15

(starts searching at
position 3)

(not found)

Methods — Equality

boolean b = word1.**equals**(word2);

returns **true** if the string **word1** is equal to **word2**

boolean b = word1.**equalsIgnoreCase**(word2);

returns **true** if the string **word1** matches **word2**, case-blind

```
b = "Raiders".equals("Raiders");//true  
b = "Raiders".equals("raiders");//false  
b = "Raiders".equalsIgnoreCase("raiders");//true
```

```
if(team.equalsIgnoreCase("raiders"))  
    System.out.println("Go You " + team);
```

Methods — Comparisons

```
int diff = word1.compareTo(word2);
```

returns the “difference” **word1 - word2**

```
int diff = word1.compareToIgnoreCase(word2);
```

returns the “difference” **word1 - word2**,
case-blind

Usually programmers don't care what the numerical “difference” of **word1 - word2** is, just whether the difference is negative (word1 comes before word2), zero (word1 and word2 are equal) or positive (word1 comes after word2). Often used in conditional statements.

```
if(word1.compareTo(word2) > 0){  
    //word1 comes after word2...  
}
```

Examples

//negative differences

```
diff = "apple".compareTo("berry");//a before b
```

```
diff = "Zebra".compareTo("apple");//Z before a
```

```
diff = "dig".compareTo("dug");//i before u
```

```
diff = "dig".compareTo("digs");//dig is shorter
```

//zero differences

```
diff = "apple".compareTo("apple");//equal
```

```
diff = "dig".compareToIgnoreCase("DIG");//equal
```

//positive differences

```
diff = "berry".compareTo("apple");//b after a
```

```
diff = "apple".compareTo("Apple");//a after A
```

```
diff = "BIT".compareTo("BIG");//T after G
```

```
diff = "huge".compareTo("hug");//huge is longer
```

Methods — trim

String word2 = word1.**trim**();

returns a new string formed from **word1** by removing
white space at both ends
does not affect whites space in the middle

```
String word1 = “ Hi Bob “;
```

```
String word2 = word1.trim();
```

```
//word2 is “Hi Bob” – no spaces on either end
```

```
//word1 is still “ Hi Bob “ – with spaces
```

Methods — replace

String word2 = word1.**replace**(oldCh, newCh);
returns a new string formed from **word1** by
replacing all occurrences of **oldCh** with **newCh**

```
String word1 = "rare";  
String word2 = "rare".replace('r', 'd');  
//word2 is "dade", but word1 is still "rare"
```

Methods — Changing Case

```
String word2 = word1.toUpperCase();  
String word3 = word1.toLowerCase();
```

returns a new string formed from **word1** by converting its characters to upper (lower) case

```
String word1 = "HeLLo";  
String word2 = word1.toUpperCase();//"HELLO"  
String word3 = word1.toLowerCase();//"hello"  
//word1 is still "HeLLo"
```


Replacement

Example: to “convert” word1 to upper case, replace the reference with a new reference.

```
word1 = word1.toUpperCase();
```

A common bug:

```
word1.toUpperCase();
```

word1
remains
unchanged

Numbers to Strings

Three ways to convert a number into a string:

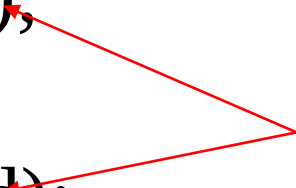
1. `String s = "" + num;`

```
s = "" + 123; //"123"
```

2. `String s = Integer.toString (i);`

3. `String s = Double.toString (d);`

4. `String s = String.valueOf (num);`



```
s =  
Integer.toString(123); //"123"  
s = Double.toString(3.14);  
 //"3.14"
```

```
s =  
String.valueOf(123); //"123"  
"
```